ALGORITMOS Y PROGRAMACIÓN II

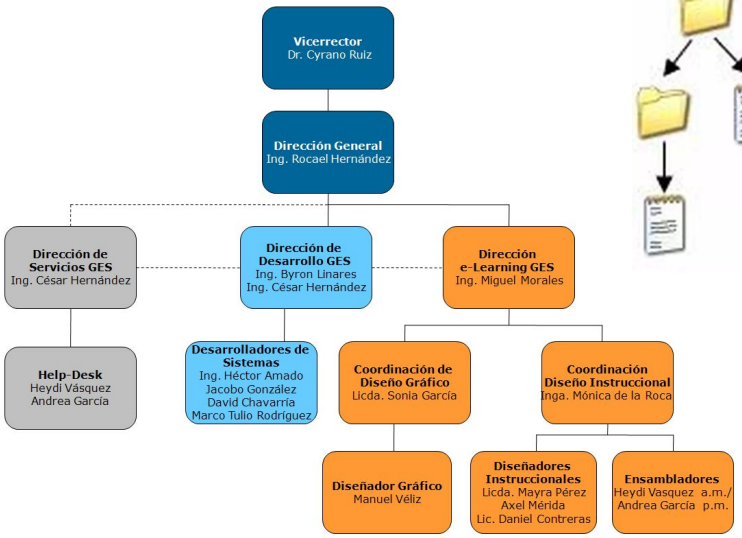
Clase 7
Árboles binarios de búsqueda

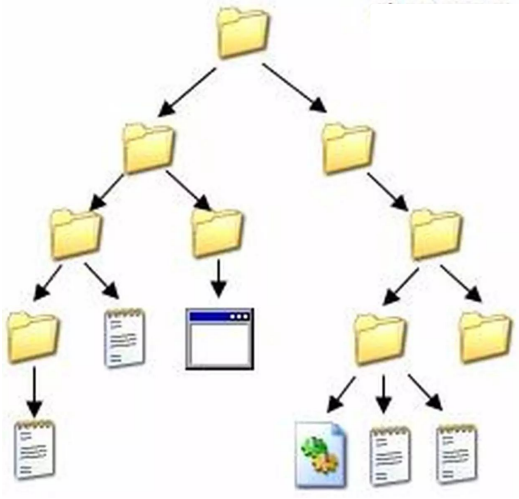
Temas de la clase

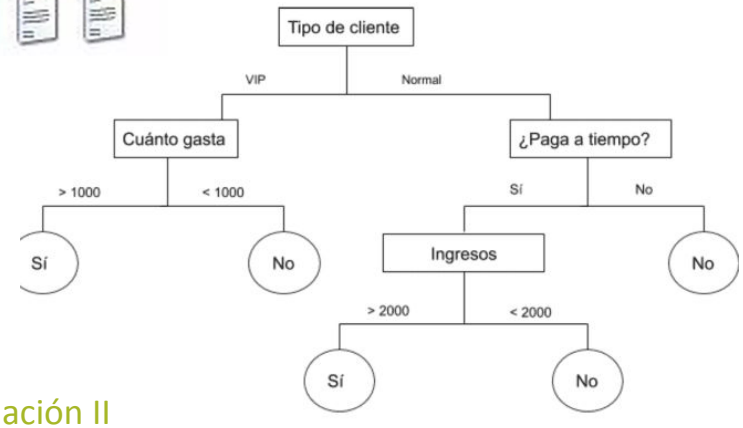
- Estructura de datos Árbol
- Árboles binarios de búsqueda (ABB)

Estructura de datos árbol

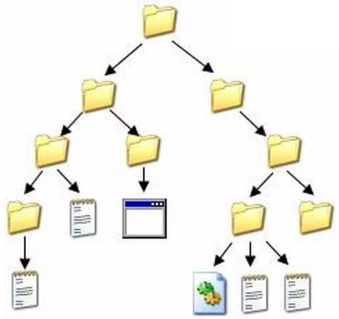
Un árbol es una estructura de datos que permite almacenar información jerárquica, o en casos donde no existe una linealidad en los datos.
Por naturaleza, los árboles son estructuras recursivas.







Árboles



CARACTERÍSTICAS

Homogénea

- Los elementos son todos del mismo tipo

Dinámica

- El tamaño puede cambiar durante la ejecución.

No-lineal

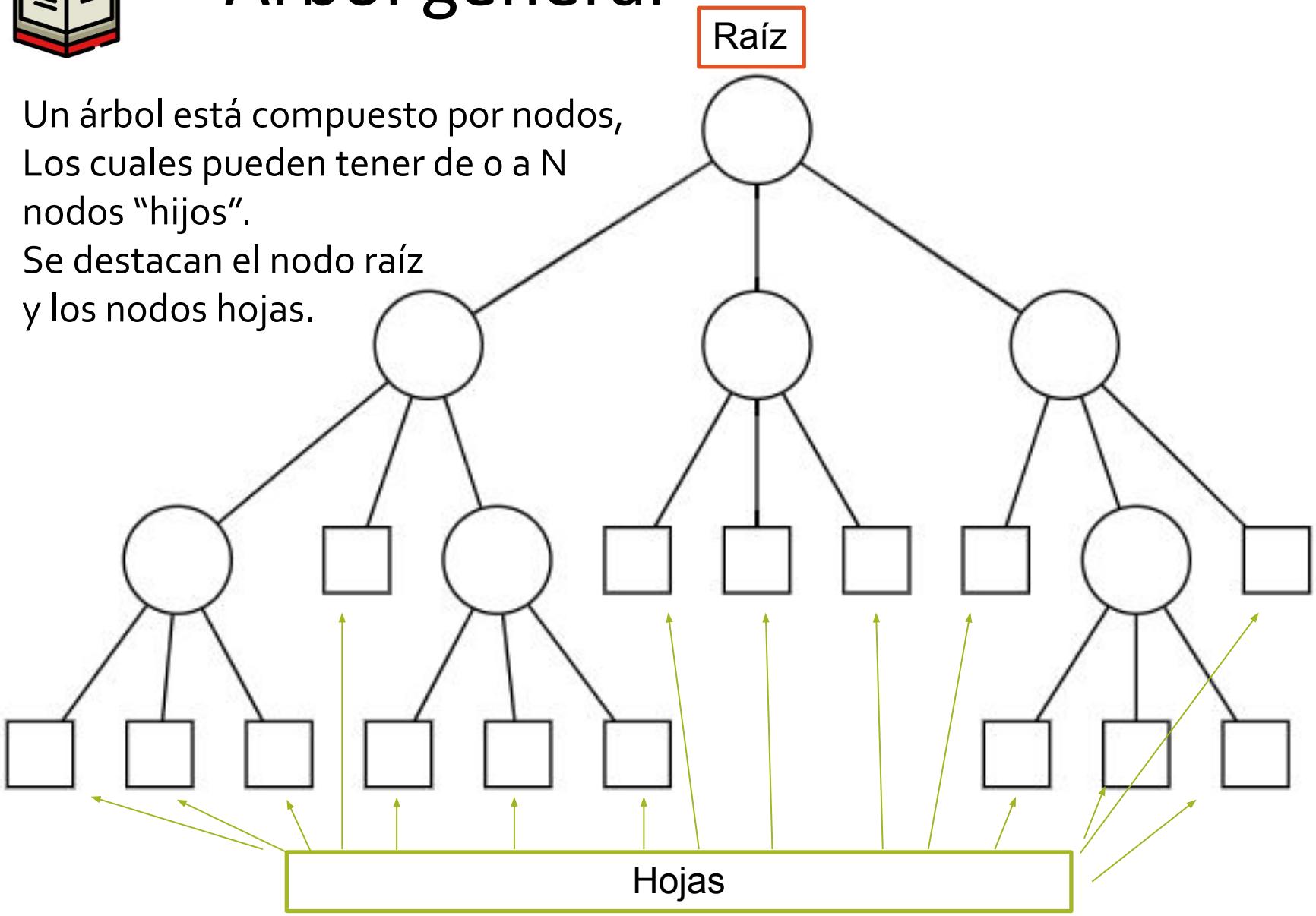
- Cada elemento tiene un antecesor y puede tener cero o más sucesores (hijos)

Secuencial

- El acceso a cada elemento es de manera secuencial

Árbol general

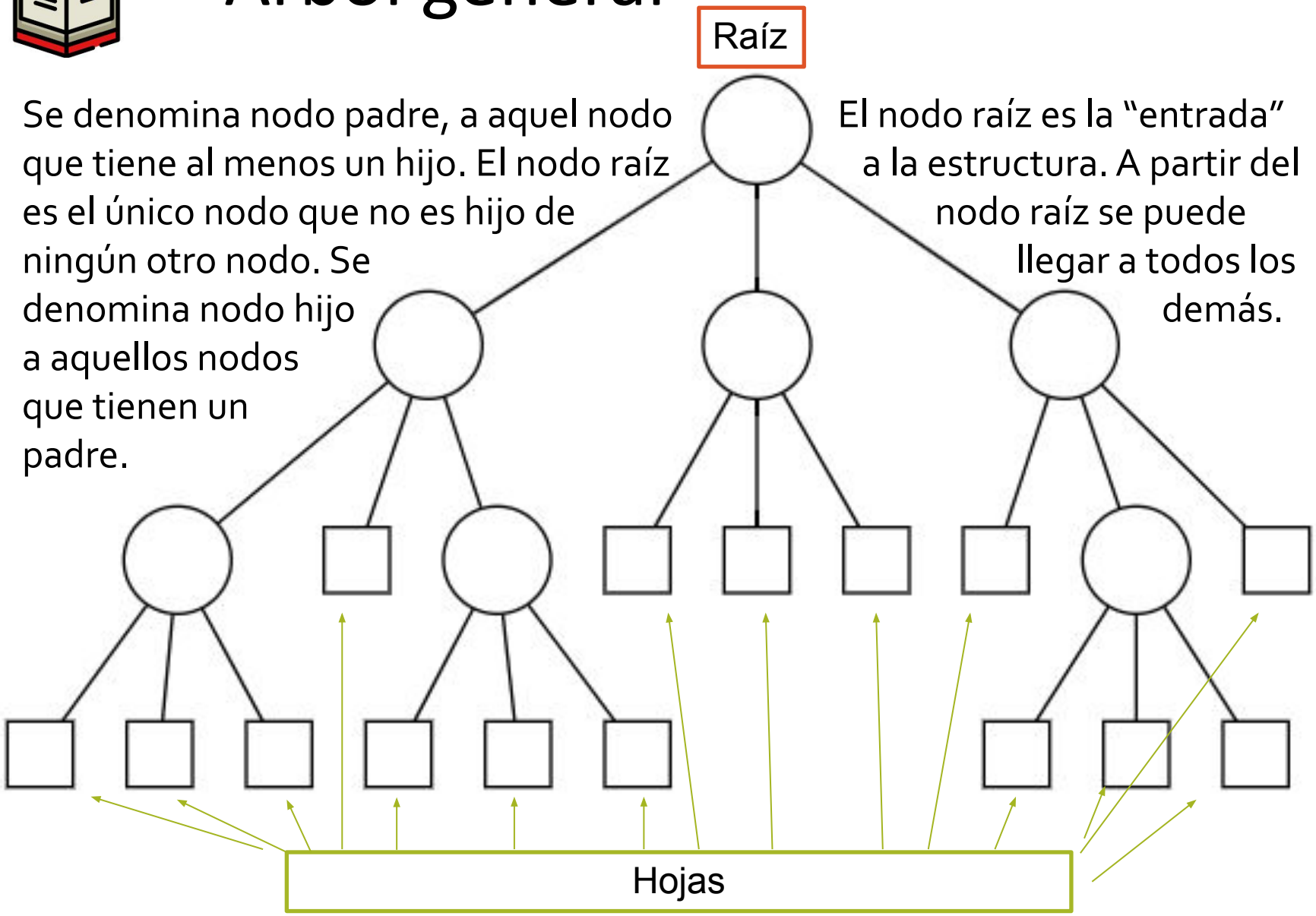
Un árbol está compuesto por nodos,
Los cuales pueden tener de 0 a N
nodos “hijos”.
Se destacan el nodo raíz
y los nodos hojas.



Árbol general

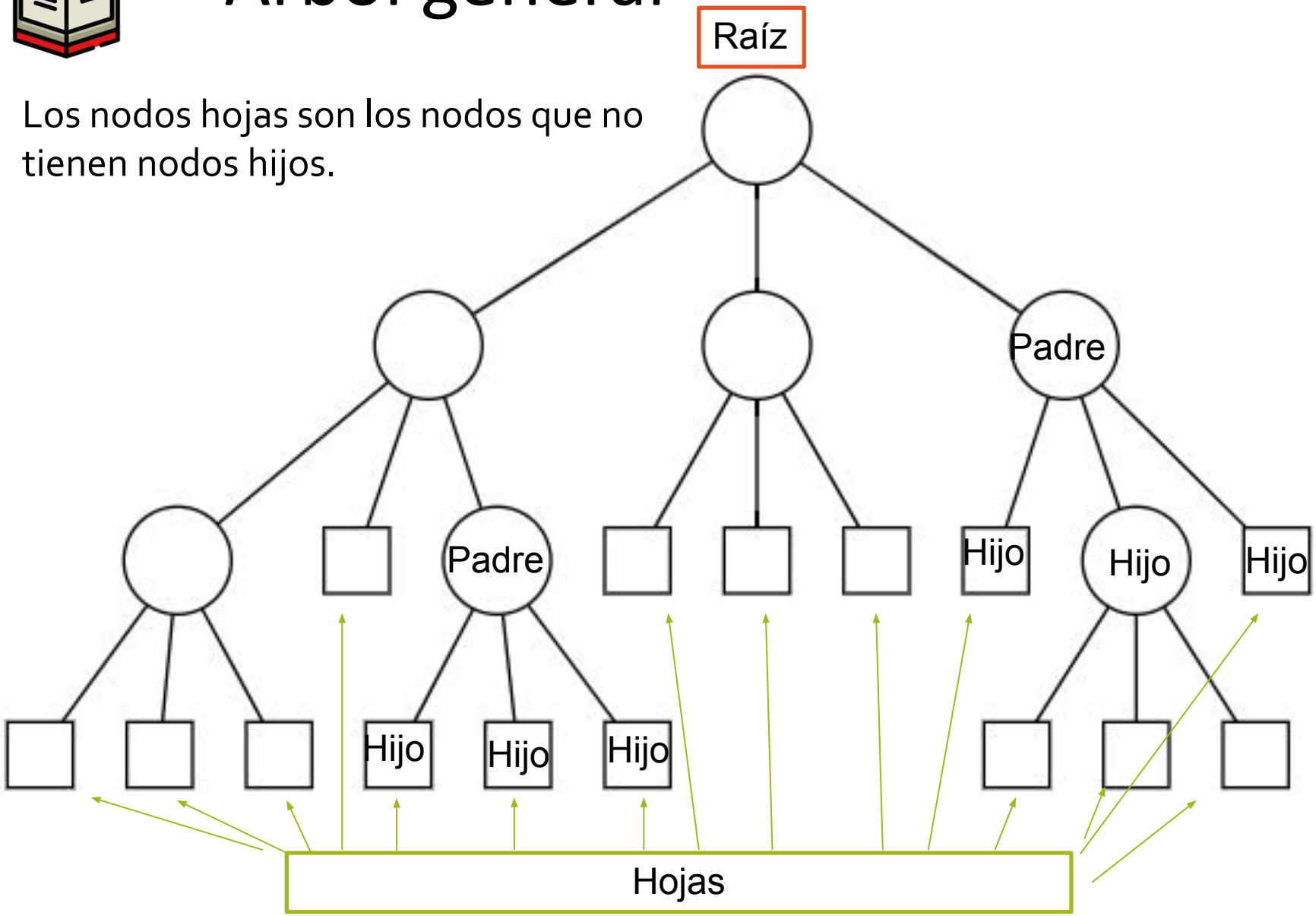
Se denomina nodo padre, a aquel nodo que tiene al menos un hijo. El nodo raíz es el único nodo que no es hijo de ningún otro nodo. Se denomina nodo hijo a aquellos nodos que tienen un padre.

El nodo raíz es la "entrada" a la estructura. A partir del nodo raíz se puede llegar a todos los demás.



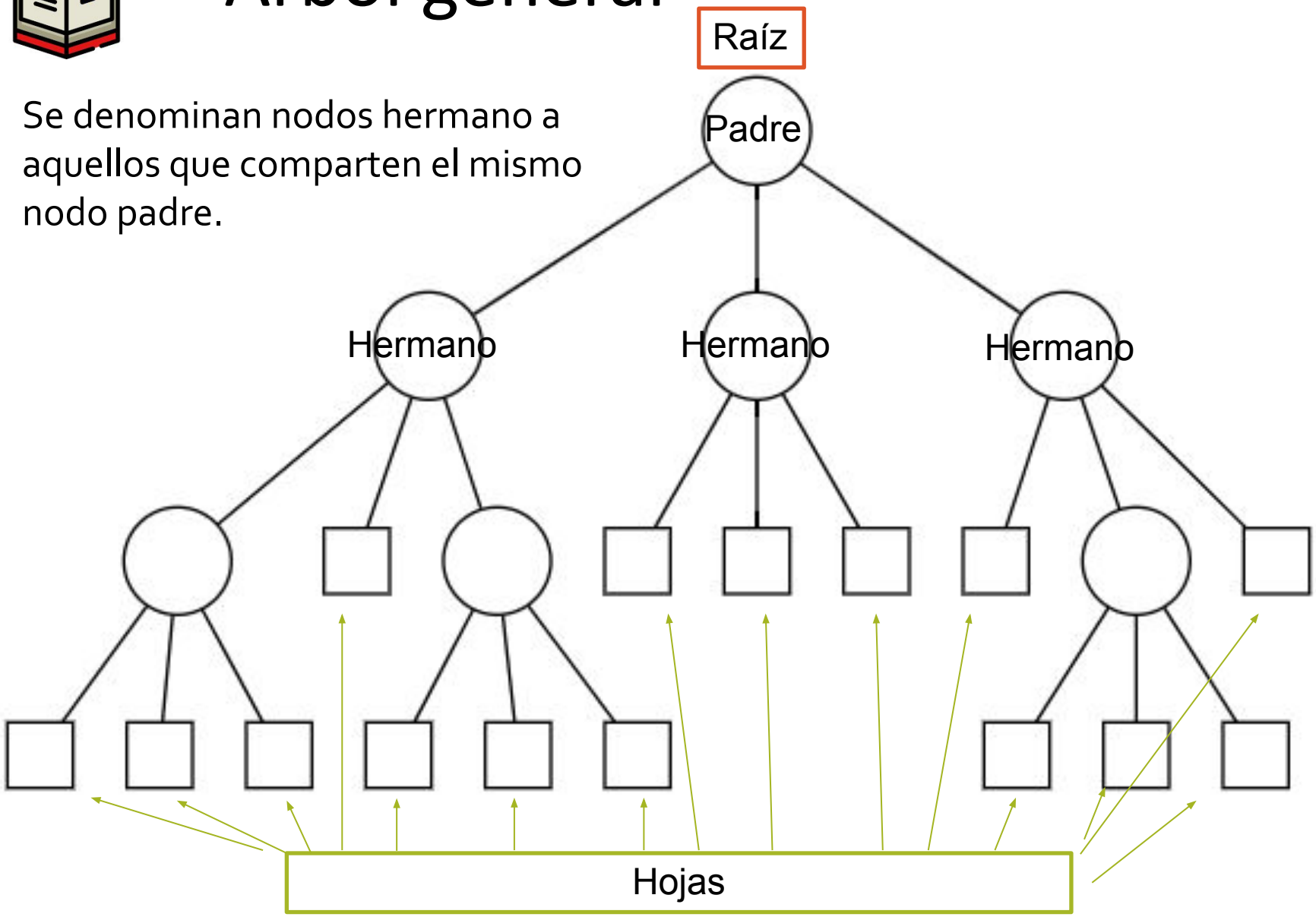
Árbol general

Los nodos hojas son los nodos que no tienen nodos hijos.



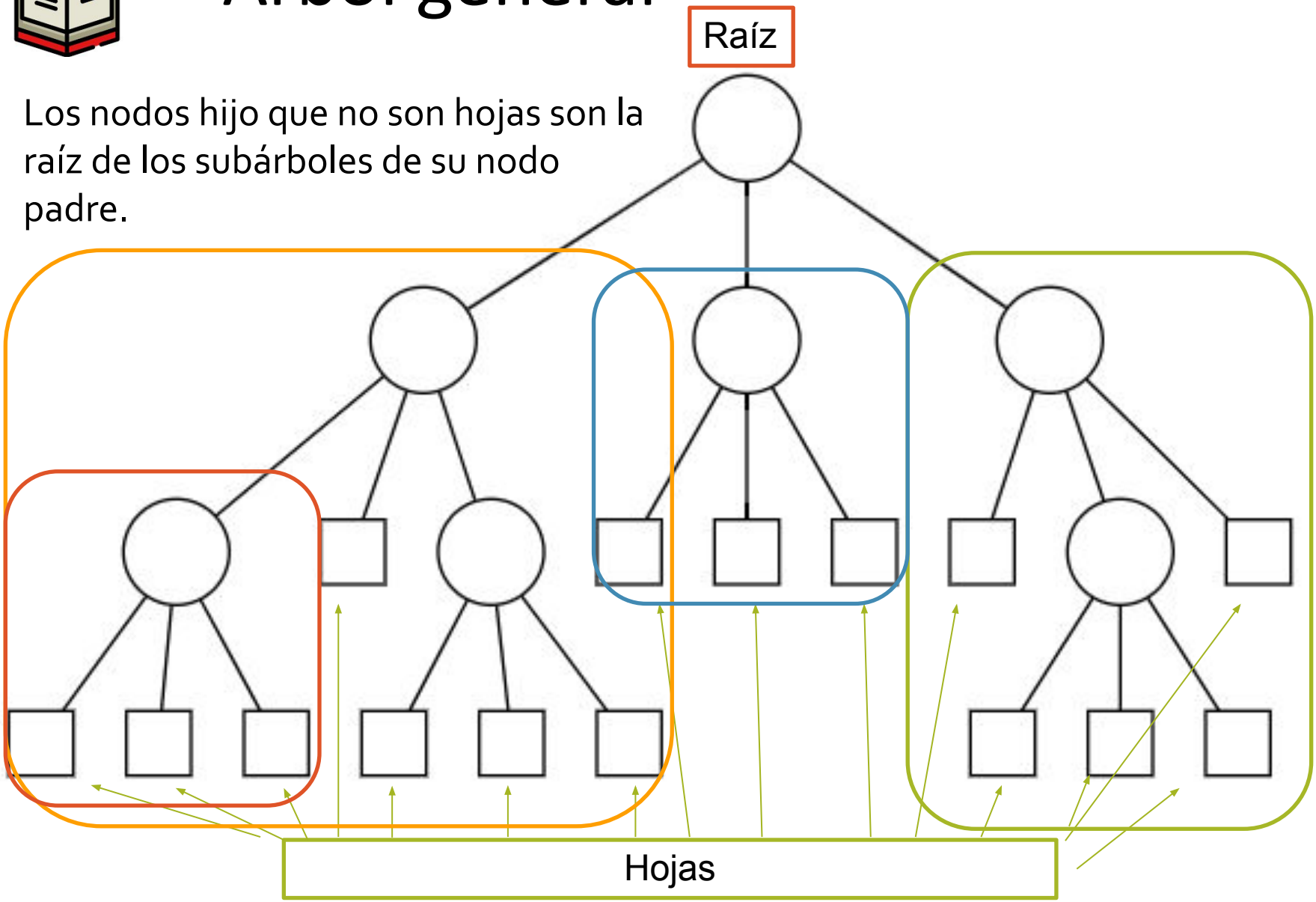
Árbol general

Se denominan nodos hermano a aquellos que comparten el mismo nodo padre.



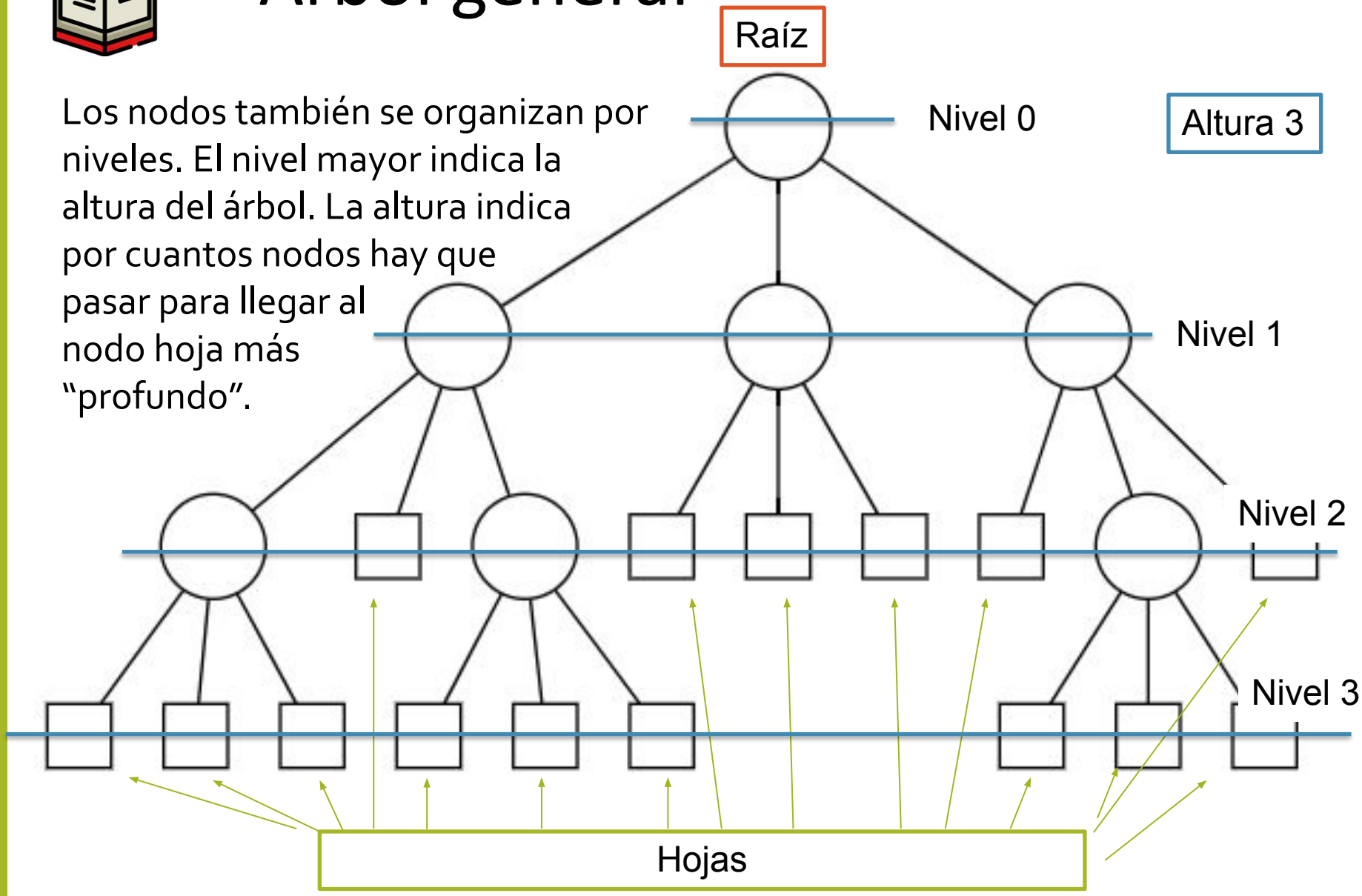
Árbol general

Los nodos hijo que no son hojas son la raíz de los subárboles de su nodo padre.



Árbol general

Los nodos también se organizan por niveles. El nivel mayor indica la altura del árbol. La altura indica por cuantos nodos hay que pasar para llegar al nodo hoja más "profundo".



Árboles

Hay muchos tipos de árboles, dependiendo de para qué problema quiera utilizarse:

- Árboles generales
- Árboles B
- Árboles B+
- Árboles B*
- Árboles rojo-negro
- Árboles AVL

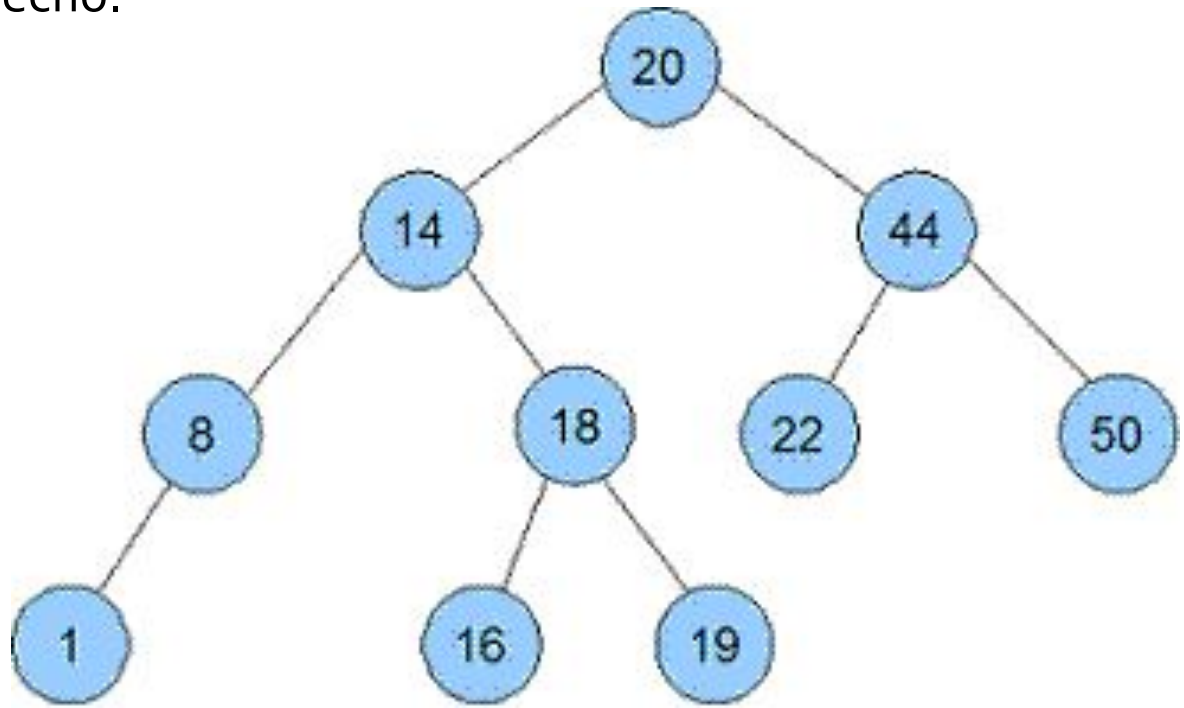
En este curso con centraremos en un tipo particular de árboles.

- Árboles binarios de búsqueda (ABB)

ABB

Los Árboles binarios de búsqueda (ABB) son un tipo particular de árboles que se caracterizan por:

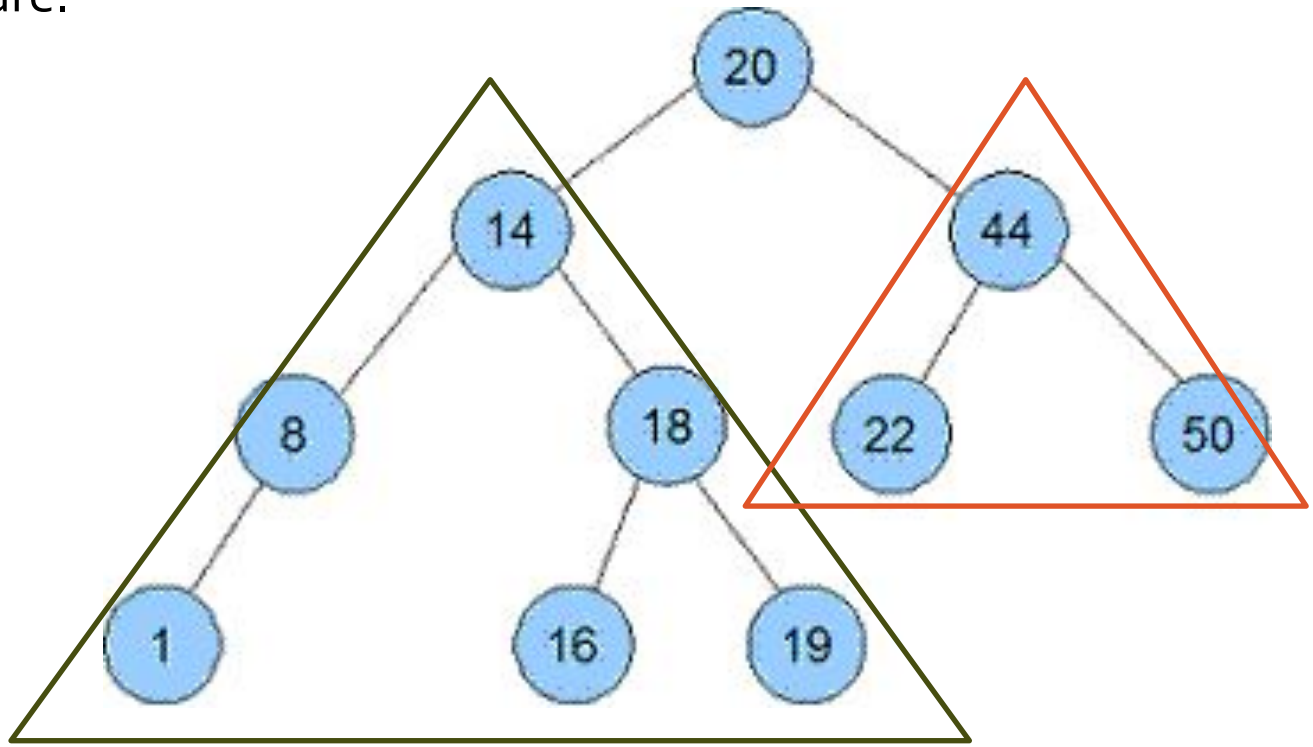
- tener sus nodos con un criterio de orden.
- cada nodo puede tener como máximo dos hijos, llamados hijo izquierdo e hijo derecho.



ABB

Dado un nodo cualquiera deben cumplirse dos condiciones:

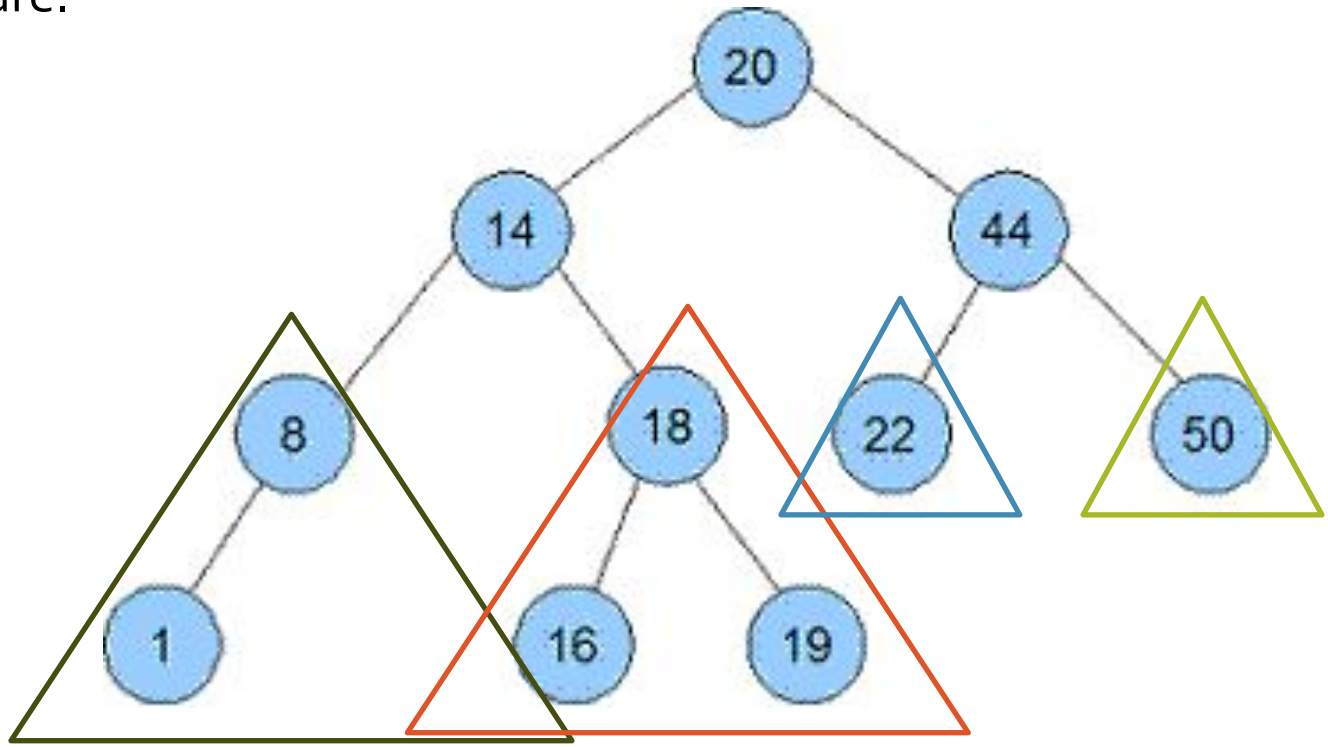
- Todos los nodos del subárbol izquierdo tienen datos menores que el nodo padre.
- Todos los nodos del subárbol derecho tienen datos mayores que el nodo padre.



ABB

Dado un nodo cualquiera deben cumplirse dos condiciones:

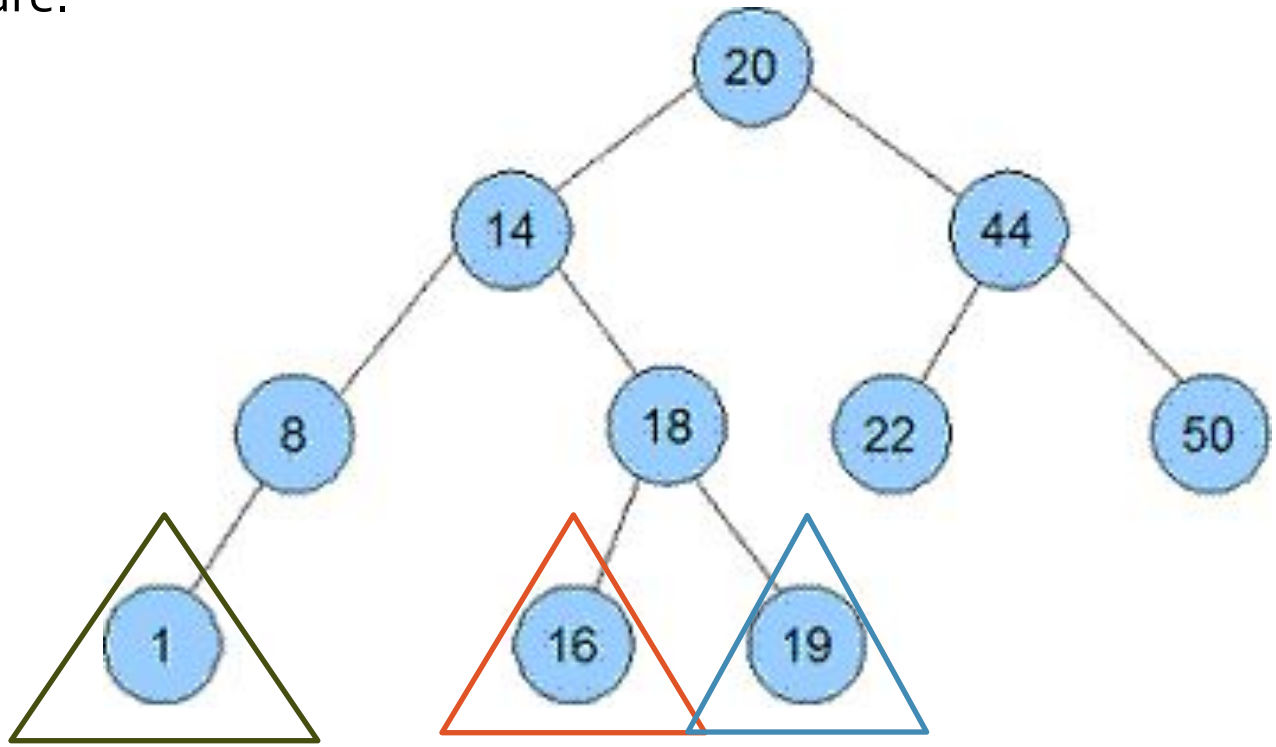
- Todos los nodos del subárbol izquierdo tienen datos menores que el nodo padre.
- Todos los nodos del subárbol derecho tienen datos mayores que el nodo padre.



ABB

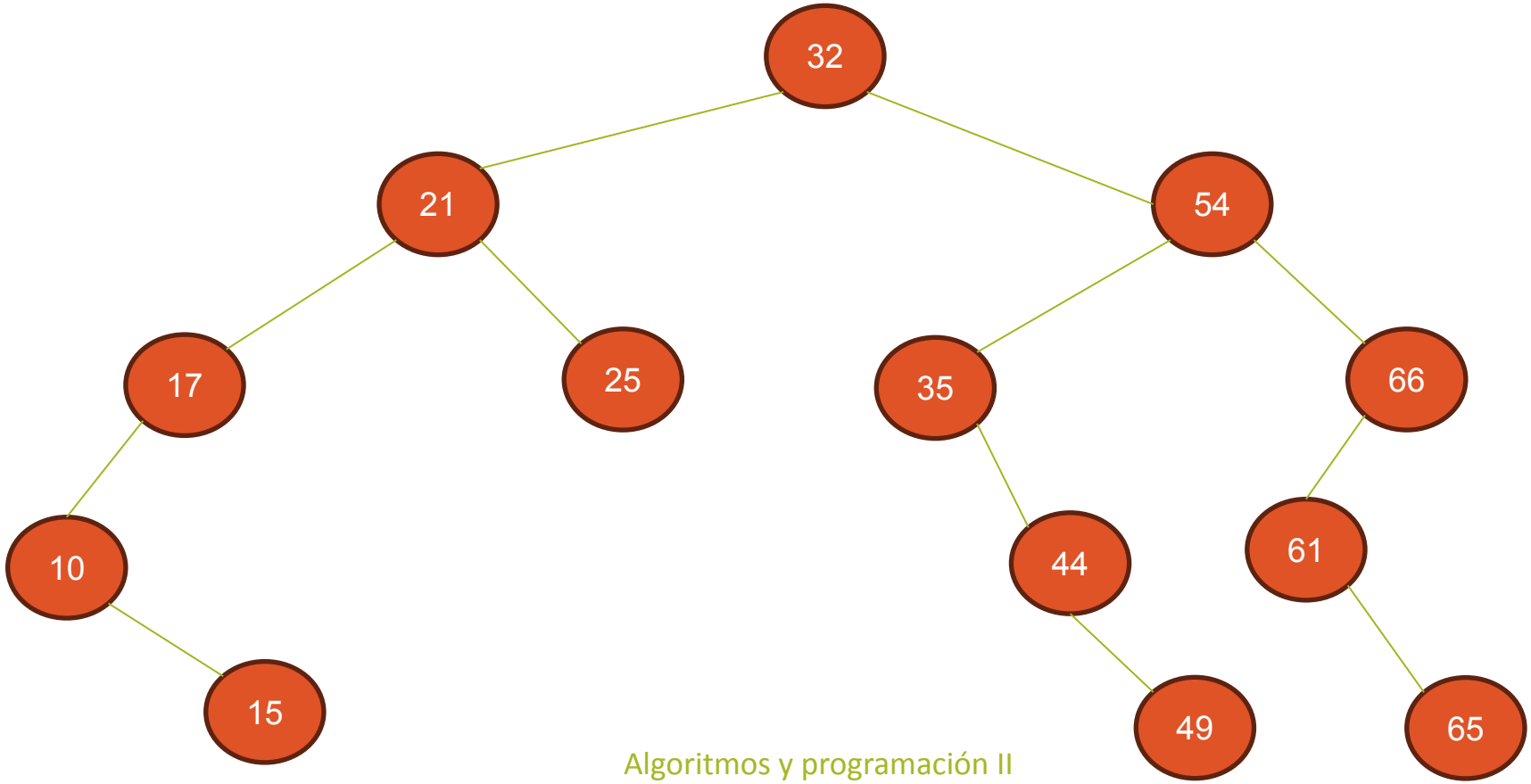
Dado un nodo cualquiera deben cumplirse dos condiciones:

- Todos los nodos del subárbol izquierdo tienen datos menores que el nodo padre.
- Todos los nodos del subárbol derecho tienen datos mayores que el nodo padre.



Armando el árbol

¿Cómo se construye un árbol ante la llegada de la siguiente serie de números?
32 – 21 – 54 – 17 – 35 – 44 – 10 – 25 – 66 – 49 – 61 – 15 – 65



Armando el árbol

Agregando nodos al árbol.

```
program
uses GenericABB;

type
  ABBInteger = specialize ABB<integer>;

var
  arbol: ABBInteger;

begin
arbol:= ABBInteger.create();
agregarNodo(arbol, 32);
agregarNodo(arbol, 21);
agregarNodo(arbol, 54);
...
end.
```

Armando el árbol

Agregando nodos al árbol.

```
procedure agregarNodo(a: ABBInteger; i: integer);
      begin
      if a.isEmpty() then
          a.insertCurrent(i)
      else
      if i < a.current() then
          agregarNodo(a.getLeftChild(), i)
      else
          agregarNodo(a.getRightChild(), i);
      end;
```

El caso base es cuando llegamos al árbol vacío. Ya sea porque es el primer elemento que se agrega, o porque recursivamente se llegó a una hoja (cuyos hijos son árboles vacíos)

Armando el árbol

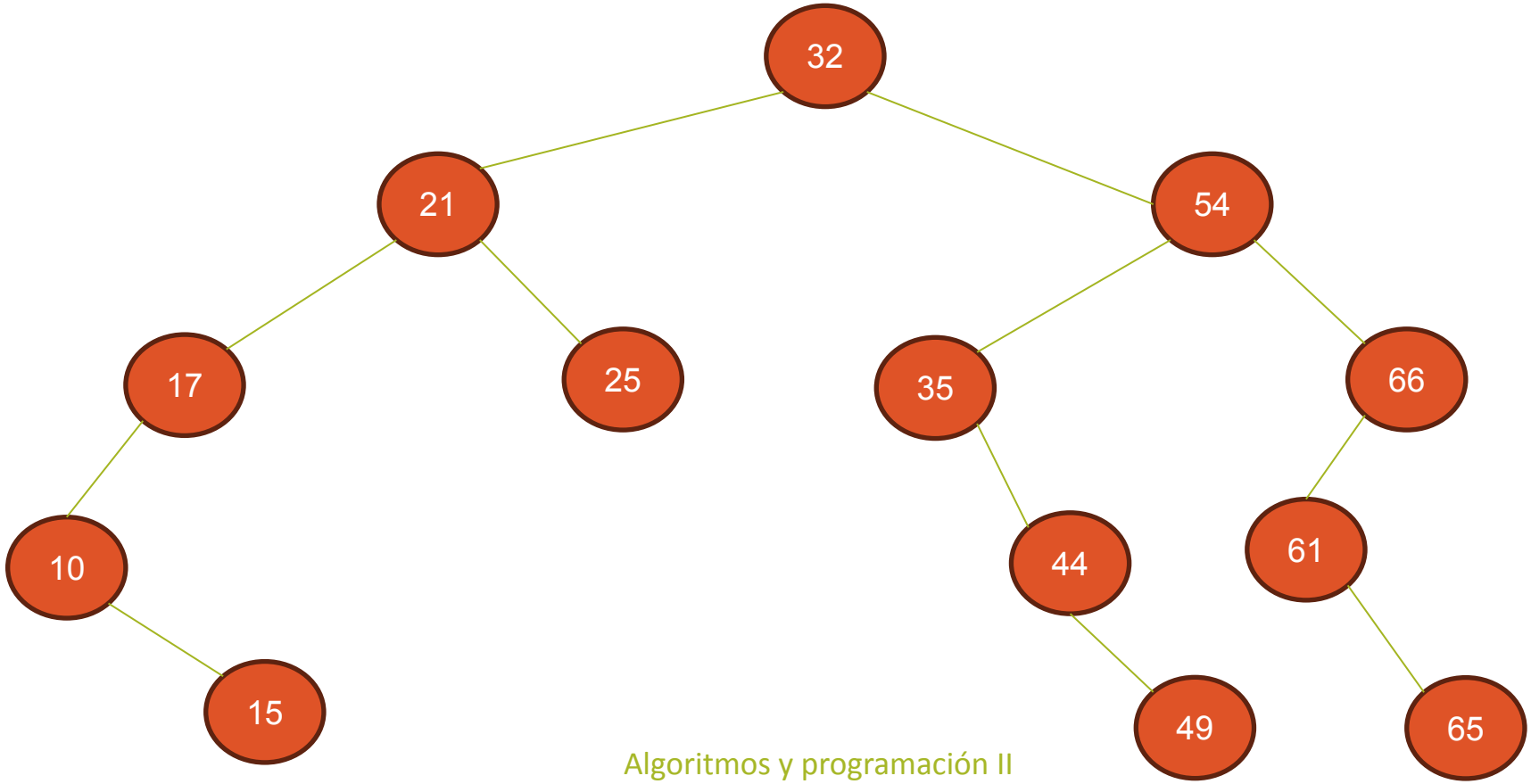
Agregando nodos al árbol.

```
procedure agregarNodo(a: ABBInteger; i: integer);
      begin
      if a.isEmpty() then
          a.insertCurrent(i)
      else
      if i < a.current() then
          agregarNodo(a.getLeftChild(), i)
      else
          agregarNodo(a.getRightChild(), i);
      end;
```

Según si el elemento a insertar es mayor o menor que el nodo que estamos visitando, invocamos al procedimiento recursivamente, pidiéndole al mismo nodo su subárbol izquierdo o su subárbol derecho.

Buscando un dato

¿Cómo se busca un dato en un ABB?
¿Cómo busco el 44?

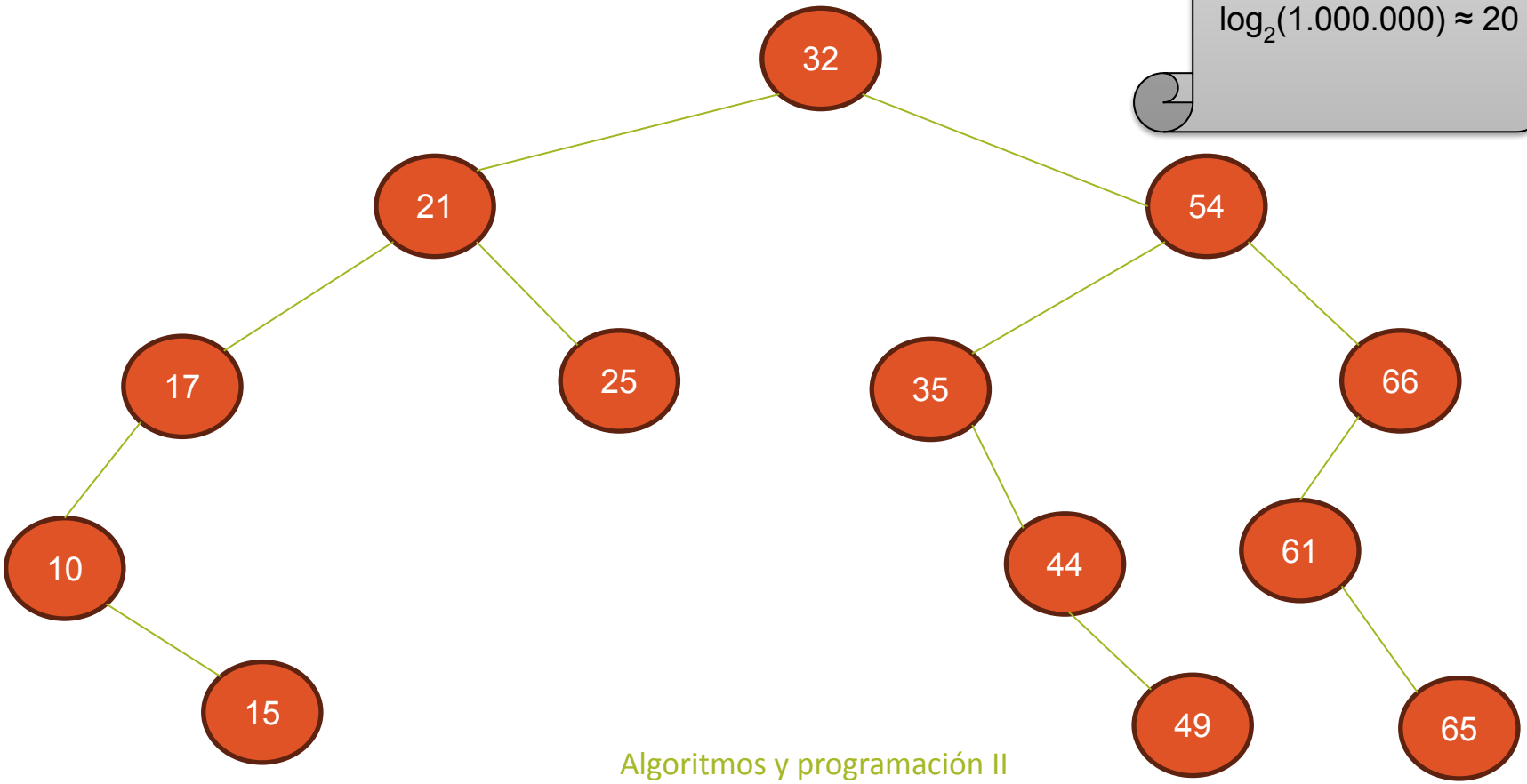


Buscando un dato

¿Cómo nos damos cuenta que el dato no existe en el ABB?
¿Cómo busco el 30?

En promedio el tiempo de búsqueda de cualquier elemento es del orden $\log_2(N)$

$\log_2(1.000.000) \approx 20$



Buscando en el árbol

Buscando un dato.

```
function buscarUnElemento(a: ABBInteger; i: integer): boolean;
  begin
  if a.isEmpty() then
      buscarUnElemento:= false
  else
  if a.current() = i then
      buscarUnElemento:= true
  else
  if i < a.current() then
      buscarUnElemento:= buscarUnElemento(a.getLeftChild(), i)
  else
      buscarUnElemento:= buscarUnElemento(a.getRightChild(), i);
  end;
```

Actividades en Máquina

ACTIVIDAD 1: Crear el archivo `ABB.pas`

Cree un programa que:

- cree un ABB.
- le agregue 20 valores al azar entre 1 y 30 (imprima por consola los valores que se van agregando para poder chequear el siguiente ítem).
- Para el rango 1..30 haga la búsqueda de cada valor dentro del ABB informando si se encuentra o no.

¿Qué sucede con el árbol si al azar sale dos o más veces el mismo valor?

Contando elementos

Contando elementos.

```
function contarElementos(a: ABBInteger): integer;
      begin
      if a.isEmpty() then
          contarElementos:= 0
      else
          begin
          contarElementos:=  contarElementos(a.getLeftChild()) +
                             contarElementos(a.getRightChild()) +
                             1;
          end
      end;
```

Actividades en Máquina

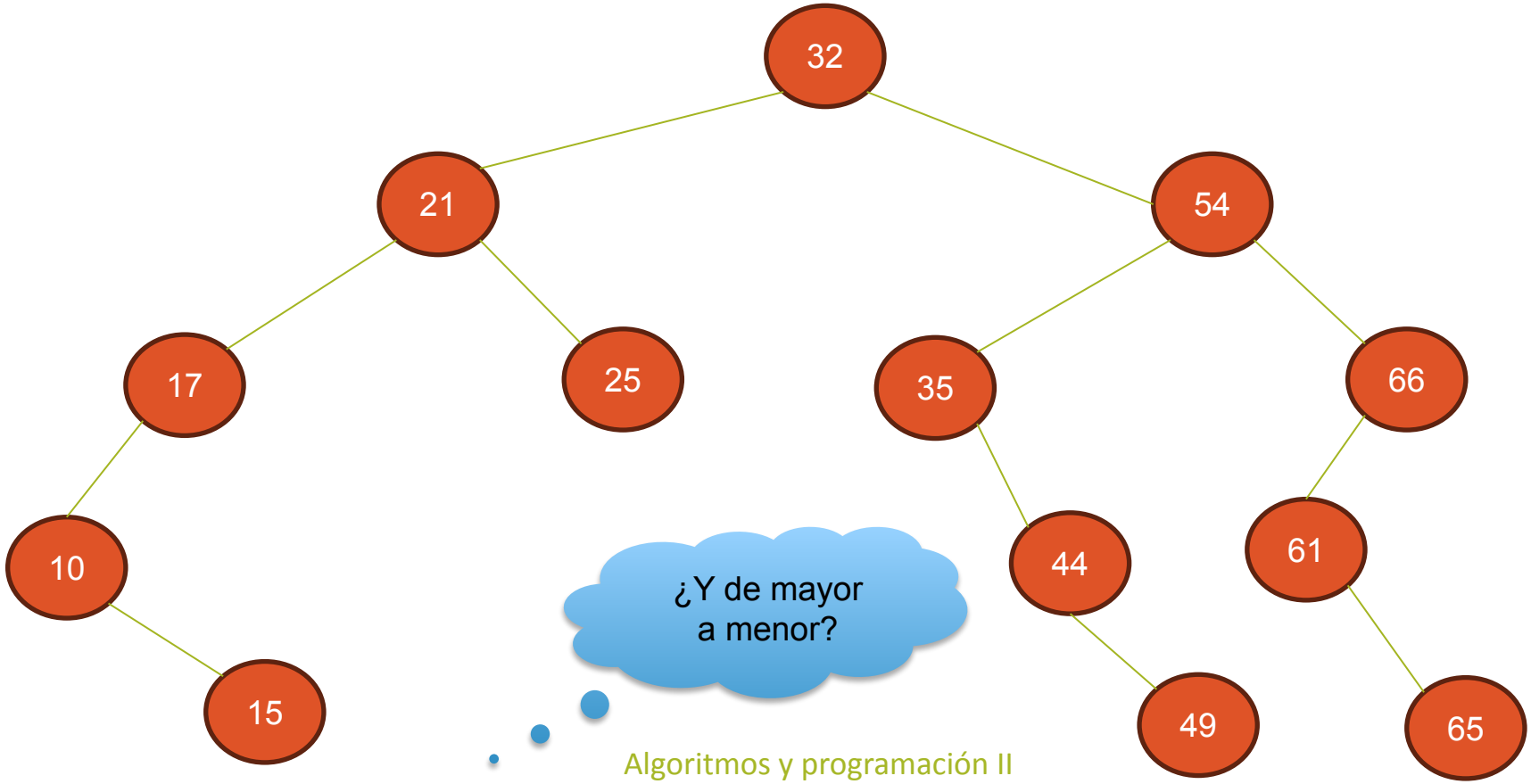
ACTIVIDAD 2: Modifique el archivo `ABB.pas`

Modifique el programa para:

- cree un ABB.
- agregue valores al azar entre 1 y 300 hasta que salga el valor 0 (cuente la cantidad de valores que se van agregando para poder chequear el siguiente ítem).
- Imprima la cantidad de elementos almacenado en el ABB contando sus elementos.

Imprimiendo el árbol

¿Cómo podemos imprimir todos los elementos del árbol de menor a mayor?
Este tipo de recorrido se lo conoce como recorrido in-orden.



Implementando el árbol

Imprimiendo los elementos in-orden.

```
procedure imprimir(a: ABBInteger);
begin
if not a.isEmpty() then
   begin
   imprimir(a.getLeftChild());
   writeln(a.current());
   imprimir(a.getRightChild());
   end;
end;
```

Actividades en Máquina

ACTIVIDAD 3: Modifique el archivo `ABB.pas`

Agregue al programa los módulos::

- imprimir(a: ABBInteger);
  // imprime los elementos del árbol de menor a mayor.

- imprimirInverso(a: ABBInteger)
  // imprime los elementos del árbol de mayor a menor.

Modifique el programa invocando a los dos módulos para comprobar el correcto funcionamiento de los mismos.

Imprimiendo el árbol

Existen otras estrategias para recorrer los elementos del árbol que son utilizadas en problemas específicos.

- Recorrido pre-orden
  - Primero se imprime el dato del nodo
  - Luego se visita el hijo izquierdo
  - Por último se visita el hijo derecho
- Recorrido post-orden
  - Primero se visita el hijo izquierdo
  - Luego se visita el hijo derecho
  - Por último se imprime el dato del nodo
- Recorrido por niveles
  - Primero se imprime el dato de la raíz
  - Luego los datos de todos los nodos de nivel 1
  - Luego los datos de todos los nodos de nivel 2
  - … así hasta imprimir todos los nodos del último nivel

Actividades en Máquina

ACTIVIDAD 4: Modifique el archivo `ABB.pas`

Agregue al programa los módulos:

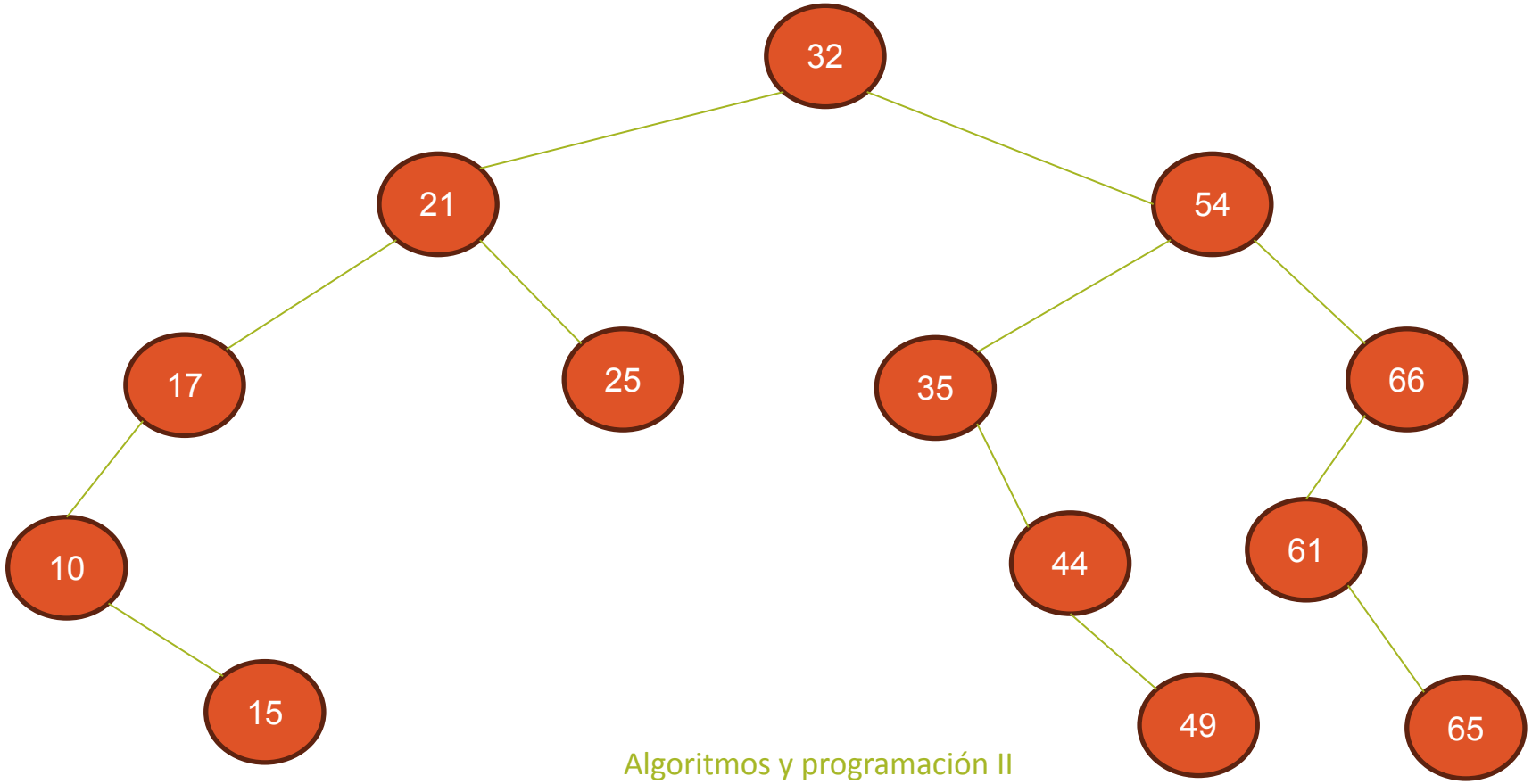
- imprimirPreOrden()
  // imprime los elementos con el recorrido pre-orden.

- imprimirPostOrden()
  // imprime los elementos con el recorrido post-orden.

Modifique el programa invocando a los dos módulos para comprobar el correcto funcionamiento de los mismos.

Buscando el mínimo y el máximo

¿Cómo buscamos el elemento más chico? ¿Y el más grande?



Buscando el valor mínimo

Buscando el mínimo.

```
function minimo(a: ABBInteger): integer;
begin
if a.hasLeftChild() then
   minimo:= minimo(a.getLeftChild())
else
   minimo:= a.current();
end;
```

Le preguntamos al nodo actual si tiene hijo izquierdo.

```
// Programa principal
var a: ABBInteger;
begin
...
if not a.isEmpty() then
   writeln(minimo(a))
else
   writeln('El árbol está vacío. No existe el valor mínimo');
end.
```

Precondición:
El árbol NO debería estar vacío.

Actividades en Máquina

ACTIVIDAD 5: Modifique el archivo `ABB.pas`

Agregue al programa los módulos:

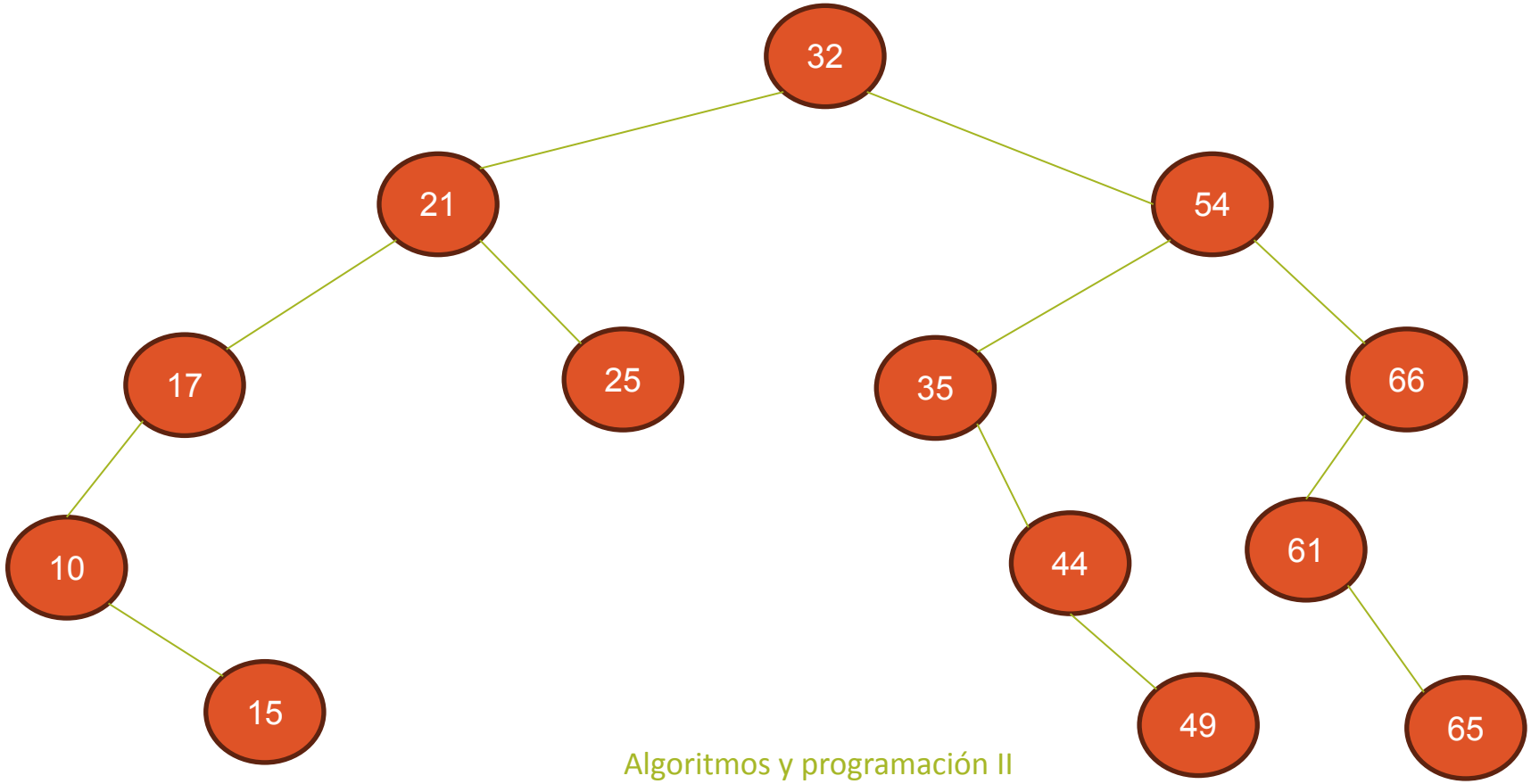
- minimo(a: ABBInteger): integer
  // Devuelve el elemento más chico del árbol

- maximo(a: ABBInteger): integer
  // Devuelve el elemento más grande del árbol.

Modifique el programa invocando a los dos módulos para comprobar el correcto funcionamiento de los mismos.

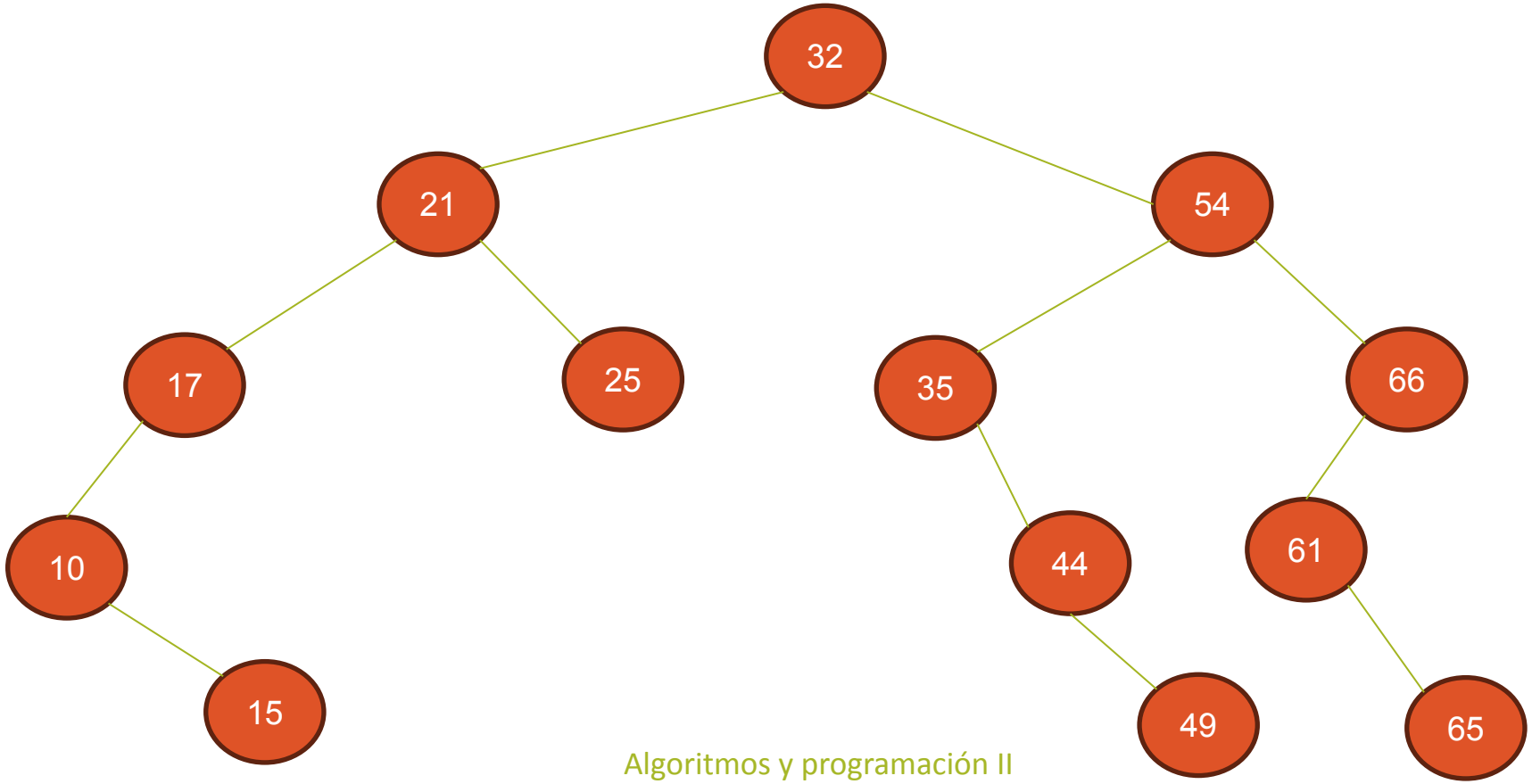
Recorriendo el árbol

¿Cómo recorremos los elementos menores a un valor?
¿Cómo hacemos para trabajar solo con los valores menores a 40?



Recorriendo el árbol

¿Cómo recorremos los elementos menores a un valor?
¿Cómo hacemos para trabajar solo con los valores mayores a 40?



Recorrido parcial

Imprimiendo elementos menores a un valor.

```
procedure imprimirMenores(a: ABBInteger; valor: integer);
begin
if not a.isEmpty() then
   begin
   if (a.current() <= valor) then
        begin
        imprimirMenores(a.getLeftChild(), valor);
        writeln(a.current);
        imprimirMenores(a.getRightChild(), valor);
        end
   else
        imprimirMenores(a.getLeftChild(), valor);
   end;
end;
```

Actividades en Máquina

ACTIVIDAD 6: Modifique el archivo `UABB.pas`

Agregue al programa los módulos:

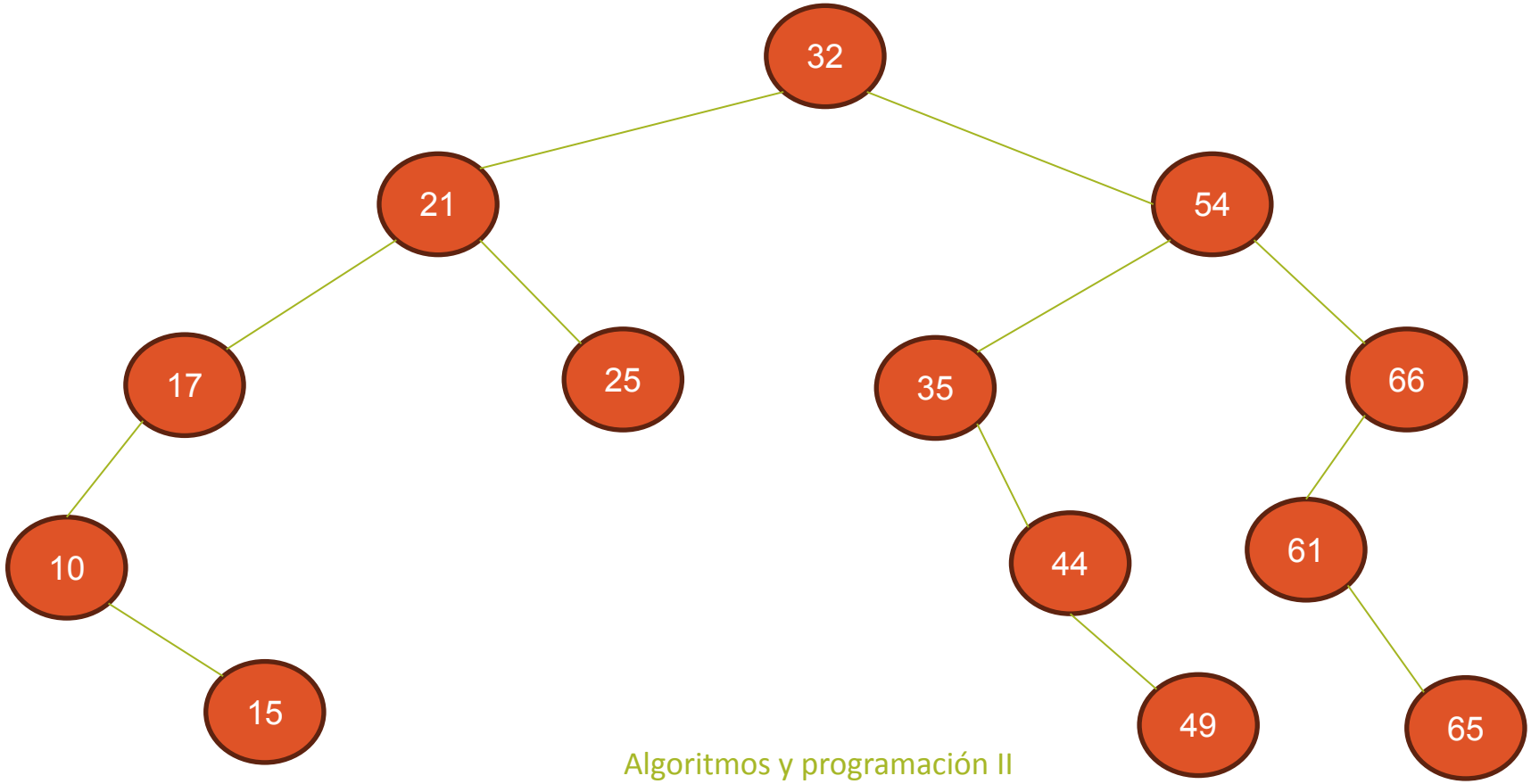
- imprimirMenores(a: ABBInteger; valor: integer)
  // imprime de menor a mayor todos los elementos del árbol que sean
  // menores o iguales al valor recibido por parámetro.

- imprimirMayores(a: ABBInteger; valor: integer)
  // imprime de menor a mayor todos los elementos del árbol que sean
  // mayores o iguales al valor recibido por parámetro.

Modifique el programa invocando a los dos módulos para comprobar el correcto funcionamiento de los mismos.

Recorriendo el árbol

¿Cómo recorremos los elementos menores a un valor y mayores a otro?
¿Cómo hacemos para trabajar con los valores mayores a 20 y menores a 45?



Recorrido parcial

Imprimiendo elementos dentro de un rango.

```
procedure imprimirAcotado(a: ABBInteger; valorMin, valorMax: integer );
begin
if not a.isEmpty() then
   begin
   if (a.current() >= valorMin) and (a.current() <= valorMax) then
        begin
        imprimirAcotado(a.getLeftChild(), valorMin, valorMax);
        writeln(a.current);
        imprimirAcotado(a.getRightChild(), valorMin, valorMax);
        end
   else
   if (a.current() <= valorMin) then
        imprimirAcotado(a.getRightChild(), valorMin, valorMax)
   else
        imprimirAcotado(a.getLeftChild(), valorMin, valorMax);
   end;
end;
```

Actividades en Máquina

ACTIVIDAD 7: Modifique el archivo `ABB.pas`

Agregue al programa el módulo:

- imprimirAcotado(a: ABBInteger; valorMin, valorMax: integer)
  // imprime de menor a mayor todos los elementos del árbol que estén
  // dentro del rango [valorMin..valorMax].

Modifique el programa invocando al módulo para comprobar el correcto funcionamiento del mismo.